

Redis 세션 관리와 Supabase 통합

1. Docker Compose로 Redis 환경 준비

구성 서비스

이 실습은 `docker-compose.yml` 로 Redis Stack과 RedisInsight를 함께 실행합니다.

서비스	컨테이너	포트	역할
<code>redis</code>	<code>redis-stack</code>	<code>6379</code>	Redis 데이터 저장 및 조회
<code>redis</code> 내장 UI	<code>redis-stack</code>	<code>8001</code>	Redis Stack에 포함된 RedisInsight

RedisInsight는 `http://localhost:8001` 사용하면 됩니다.

실행 및 상태 확인

```
docker compose up -d
```

결과 확인

 docker.desktop

PERSONAL

Ctrl+K

?

4







Sign in

—

□

×

 Containers [Give feedback](#)



















Container CPU usage ⓘ
0.12% / 2800% (28 CPUs available)

Container memory usage ⓘ
138.45MB / 15.25GB

[Show charts](#)

 ☒ Only show running containers

☐	Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
☐	> postgres	-	-	-	0%	3 minutes ago	  
☐	▼ 08_redis__supabase_	-	-	-	0.19%	2 minutes ago	  
☐	redis-stack	6dded1198695	redis/redis-stack:7.2.0-v10	6379:6379  Show all ports (2)	0.19%	2 minutes ago	  

Redis UI 접속 주소

최종 사용자 라이선스 계약 및 개인정보 설정

redisinsight

약성 코드의 자동 실행을 방지하려면 새 Workbench 플러그인을 추가할 때 신뢰할 수 있는 제작자의 파일만 사용하십시오.

☐ 권장 설정을 사용하세요

나열된 모든 옵션을 활성화하려면 선택하세요.

개인정보 설정

RedisInsight는 사용자 경험을 최적화하기 위해 타사 도구를 사용합니다. 수집된 모든 데이터는 익명화되며 사용자의 동의 없이는 어떠한 목적으로도 사용되지 않습니다.

☐ 분석 기능을 활성화합니다.

RedisInsight를 개선하는 데 도움을 주시려면 선택해 주세요. 저희는 익명화된 사용자 경험 데이터를 수집하여 버그를 수정하고 모든 사용자의 경험을 개선하는 데 사용합니다.

☐ 민감한 정보를 암호화하세요

데이터 암호화 기능은 데스크톱 애플리케이션 버전에서 사용할 수 있습니다. [여기에서](#) 다운로드하세요 .

알림

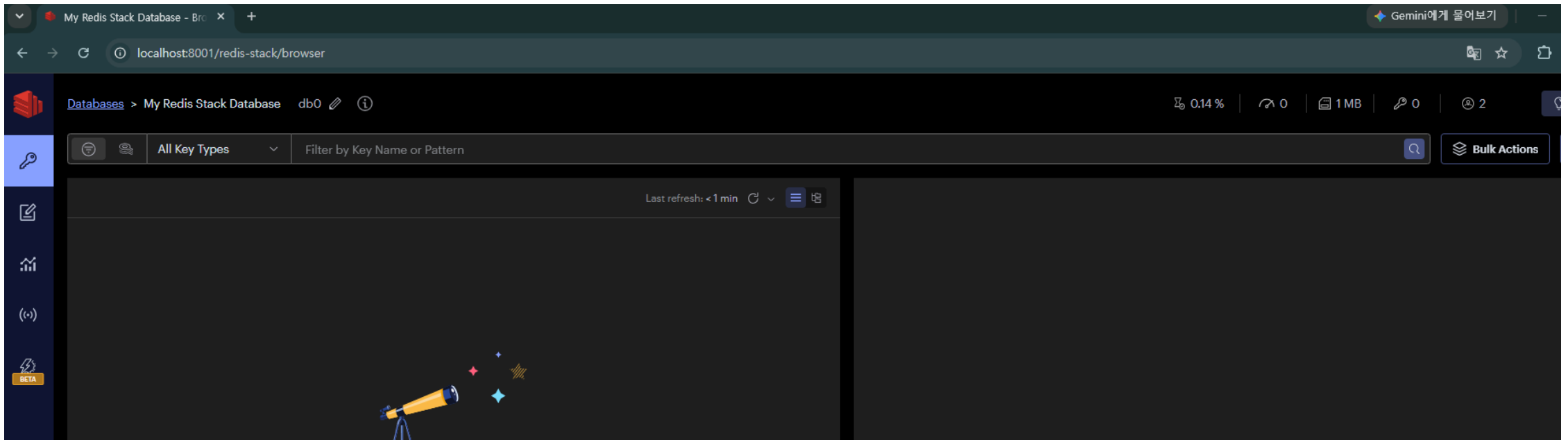
☐ 알림 표시

[RedisInsight를 사용하려면 서버 측 공개 라이선스](#) 약관에 동의해 주십시오.

☒ 저는 약관을 읽고 이해했습니다.

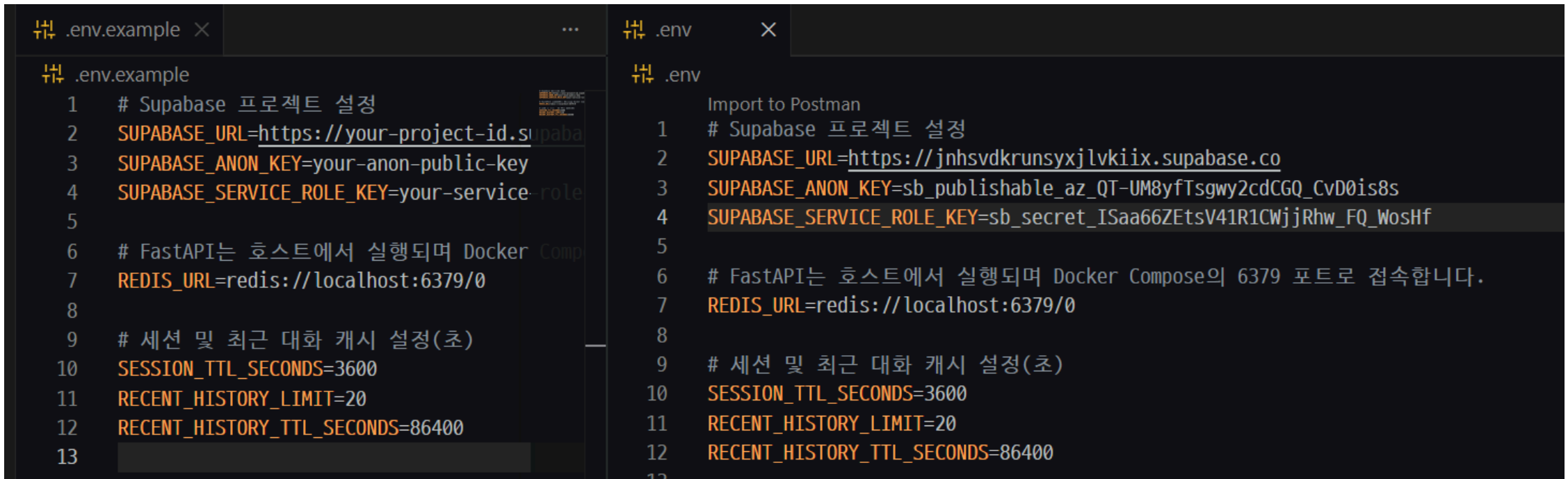
→ 제출하다

Redis UI 접속 성공



따라서 `.env` 의 설정은 다음과 같습니다.

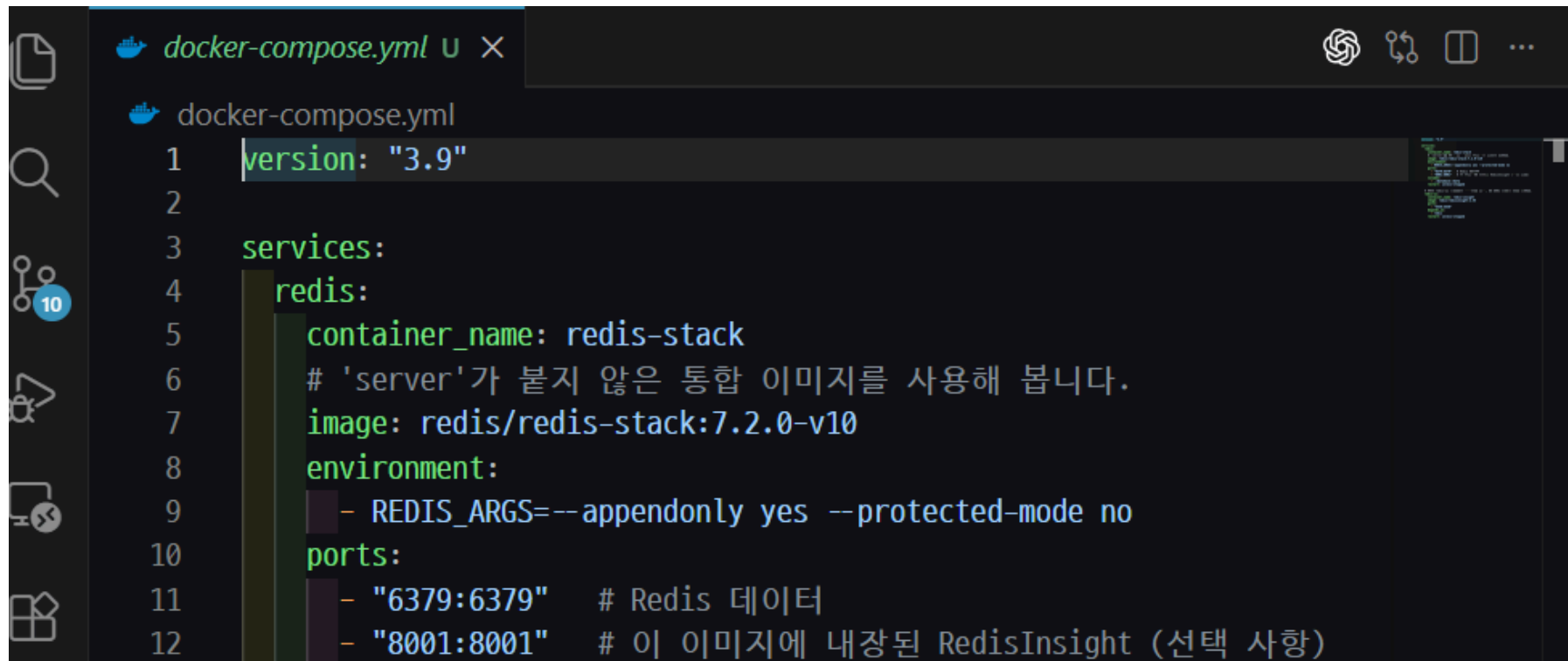
```
REDIS_URL=redis://localhost:6379/0
```



```
.env.example
1 # Supabase 프로젝트 설정
2 SUPABASE_URL=https://your-project-id.supabase.co
3 SUPABASE_ANON_KEY=your-anon-public-key
4 SUPABASE_SERVICE_ROLE_KEY=your-service-role-key
5
6 # FastAPI는 호스트에서 실행되며 Docker Compose의 6379 포트에 접속합니다.
7 REDIS_URL=redis://localhost:6379/0
8
9 # 세션 및 최근 대화 캐시 설정(초)
10 SESSION_TTL_SECONDS=3600
11 RECENT_HISTORY_LIMIT=20
12 RECENT_HISTORY_TTL_SECONDS=86400
13

.env
1 Import to Postman
2 # Supabase 프로젝트 설정
3 SUPABASE_URL=https://jnhsvdkrunsyxjlvkiix.supabase.co
4 SUPABASE_ANON_KEY=sb_publishable_az_QT-UM8yftsgwy2cdCGQ_CvD0is8s
5 SUPABASE_SERVICE_ROLE_KEY=sb_secret_ISaa66ZetsV41R1CWjjRhw_FQ_WosHf
6
7 # FastAPI는 호스트에서 실행되며 Docker Compose의 6379 포트에 접속합니다.
8 REDIS_URL=redis://localhost:6379/0
9
10 # 세션 및 최근 대화 캐시 설정(초)
11 SESSION_TTL_SECONDS=3600
12 RECENT_HISTORY_LIMIT=20
13 RECENT_HISTORY_TTL_SECONDS=86400
```

--protected-mode no 설정은 로컬 실습용입니다. 6379 포트를 외부 인터넷에 공개하는 운영 환경에서는 인증과 네트워크 접근 제어를 반드시 설정해야 합니다.



```
1 version: "3.9"
2
3 services:
4   redis:
5     container_name: redis-stack
6     # 'server'가 붙지 않은 통합 이미지를 사용해 봅니다.
7     image: redis/redis-stack:7.2.0-v10
8     environment:
9       - REDIS_ARGS=--appendonly yes --protected-mode no
10    ports:
11      - "6379:6379" # Redis 데이터
12      - "8001:8001" # 이 이미지에 내장된 RedisInsight (선택 사항)
```

FastAPI Redis 클라이언트 초기화

```
import redis

r = redis.from_url(
    "redis://localhost:6379/0",
    decode_responses=True,    # bytes → str 자동 변환
)
r.ping()    # True
```

`decode_responses=True` 가 없으면 모든 값이 `bytes` 로 반환되어 JSON 파싱·딕셔너리 비교 시 `AttributeError: 'bytes' object` 가 발생합니다.

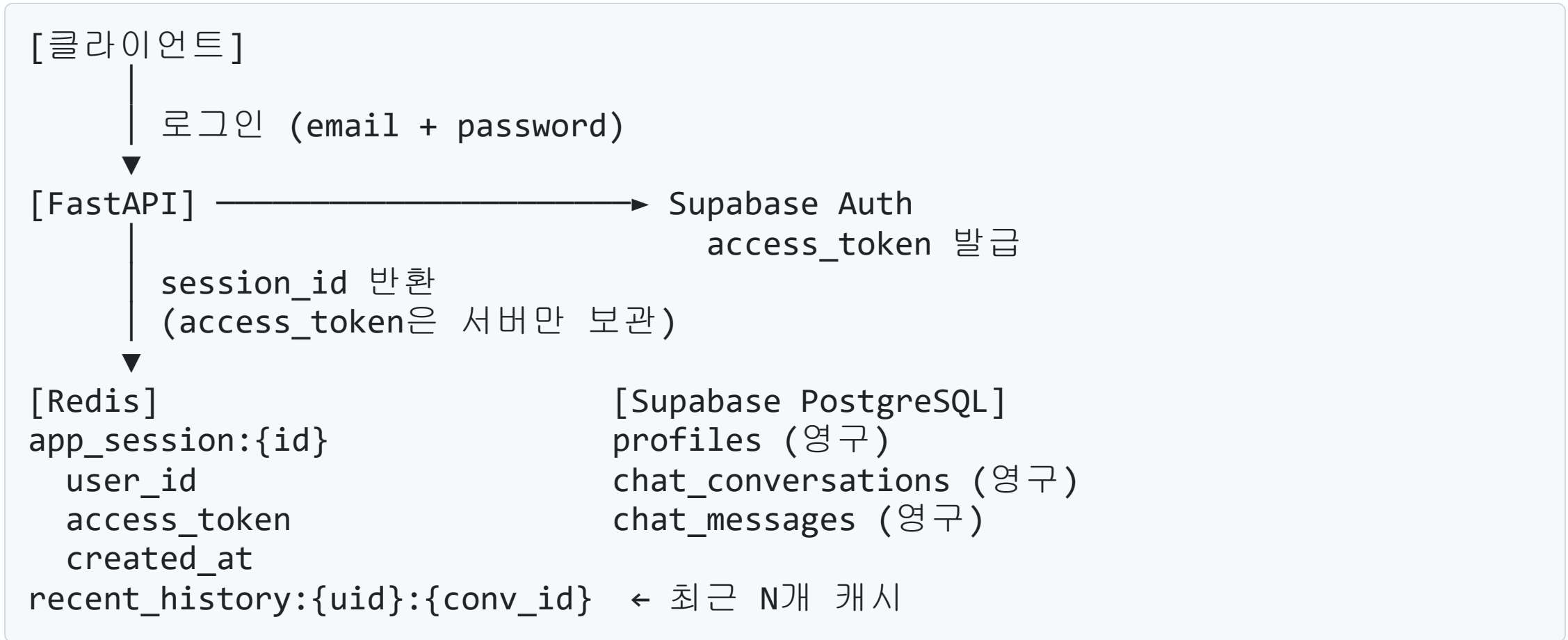
키 이름 설계

권장 패턴 : {도메인}:{식별자}

app_session:{session_id} → 앱 세션 Hash
recent_history:{user_id}:{conversation_id} → 최근 대화 캐시 List

```
KEYS app_session:*      # 키 목록 (운영 환경에서 사용 금지!)  
TYPE app_session:abc   # hash  
DEL app_session:abc    # 키 삭제
```

2. 전체 아키텍처



Supabase vs Redis 역할 분리

데이터	저장 위치	이유
회원 계정	Supabase Auth	인증 전용 시스템
프로필, 대화, 메시지	Supabase PostgreSQL	영구 보관 필요
앱 세션	Redis	TTL 자동 만료, 빠른 조회
최근 대화 캐시	Redis	LLM 컨텍스트용 빠른 읽기

3. 로그인 흐름 — Auth → 앱 세션

Supabase Auth 로그인 결과를 Redis에 저장하고 자체 `session_id` 를 발급합니다.
`access_token` 은 Redis 안에만 있으므로 클라이언트에 노출되지 않습니다.

```
@app.post("/auth/login")
def login(data: LoginRequest):
    # 1. Supabase Auth로 이메일/비밀번호 검증
    result = public_client().auth.sign_in_with_password({
        ...

    # 2. Redis에 앱 세션 저장 (access_token 포함)
    session_id = str(uuid4())
    key = f"app_session:{session_id}"
    ...

    # 3. 클라이언트에게는 session_id만 반환 (access_token 노출 안 함)
    return {"session_id": session_id, "user_id": user_id}
```

4. 이중 검증 — Redis 세션 + Supabase 토큰

Redis 세션 안의 Supabase `access_token` 도 유효한지 함께 검증합니다.

```
# 1단계: Redis 세션 존재 확인
if not data:
    raise HTTPException(401, "세션이 없거나 만료되었습니다. 다시 로그인하세요.")

# 2단계: Supabase access_token 검증
try:
    auth_result = public_client().auth.get_user(data
    ...

# 3단계: user_id 일치 확인 (변조 방지)
if str(auth_result.user.id) != data["user_id"]:
    ...

# 4단계: TTL 갱신 (Sliding Expiry)
r.expire(key, SESSION_TTL_SECONDS)
...
```

검증 단계 요약

단계	확인 내용	실패 시
1	Redis 세션 존재	401 + 재로그인 안내
2	Supabase access_token 유효성	세션 삭제 + 401
3	Redis user_id == Auth user_id	세션 삭제 + 401
4	TTL 갱신	-

5. 메시지 이중 저장 — Supabase 먼저

메시지를 영구 저장소(Supabase)와 캐시(Redis) 두 곳에 저장합니다.

저장 순서가 중요한 이유

Redis 먼저 → Supabase 실패

Redis에는 있지만 Supabase에는 없음

서버 재시작 후 캐시 만료 → 메시지 영구 유실!

Supabase 먼저 → Redis 실패

Supabase에 원본이 있으므로

POST /recent/rebuild 로 캐시 복구 가능

```

@app.post("/conversations/{conversation_id}/messages", status_code=201)
def create_message(
    conversation_id: str,
    data: MessageCreate,
    session: AppSession = Depends(get_current_session),
):
    # 소유권 확인
    get_owned_conversation(conversation_id, session.user_id)

    # 1. Supabase에 영구 저장 (반드시 성공해야 함)
    result = (
        admin_client()
        .table("chat_messages")
        ...

    # 2. Redis 캐시 저장 (실패해도 원본은 Supabase에 있음)
    cache_key = f"recent_history:{session.user_id}:{conversation_id}"
    r.rpush(cache_key, json.dumps(message, ensure_ascii=False))
    r.ltrim(cache_key, -RECENT_HISTORY_LIMIT, -1)
    r.expire(cache_key, RECENT_HISTORY_TTL_SECONDS)

    return message

```

6. 두 가지 조회 엔드포인트

엔드포인트	데이터 소스	특징
GET /conversations/{id}/messages	Supabase	전체 이력, 느릴 수 있음
GET /conversations/{id}/recent	Redis	최근 N개만, 초고속

```
@app.get("/conversations/{conversation_id}/recent")
def get_recent(
    conversation_id: str,
    session: AppSession = Depends(get_current_session),
):
    get_owned_conversation(conversation_id, session.user_id)
    key = f"recent_history:{session.user_id}:{conversation_id}"
    return [json.loads(m) for m in r.lrange(key, 0, -1)]
```

7. 캐시 복구 (Rebuild)

Redis 캐시가 만료되거나 `./database` 데이터가 유실되면, Supabase의 영구 데이터로 캐시를 다시 채웁니다.

```
@app.post("/conversations/{conversation_id}/recent/rebuild")
def rebuild_recent(conversation_id: str, session: AppSession = Depends(get_current_session),
):
    get_owned_conversation(conversation_id, session.user_id)

    # 1. Supabase에서 최근 메시지 조회
    result = (
        ...
    )
    messages = list(reversed(result.data)) # 오래된 것부터 순서 복원

    # 2. Redis 캐시 재생성
    cache_key = f"recent_history:{session.user_id}:{conversation_id}"
    r.delete(cache_key)
    ...
```

왜 reversed가 필요한가

Supabase에서 최신순으로 조회:

```
order("created_at", desc=True) → [msg5, msg4, msg3, msg2, msg1]
```

Redis List는 오래된 것부터 순서대로 저장해야 함:

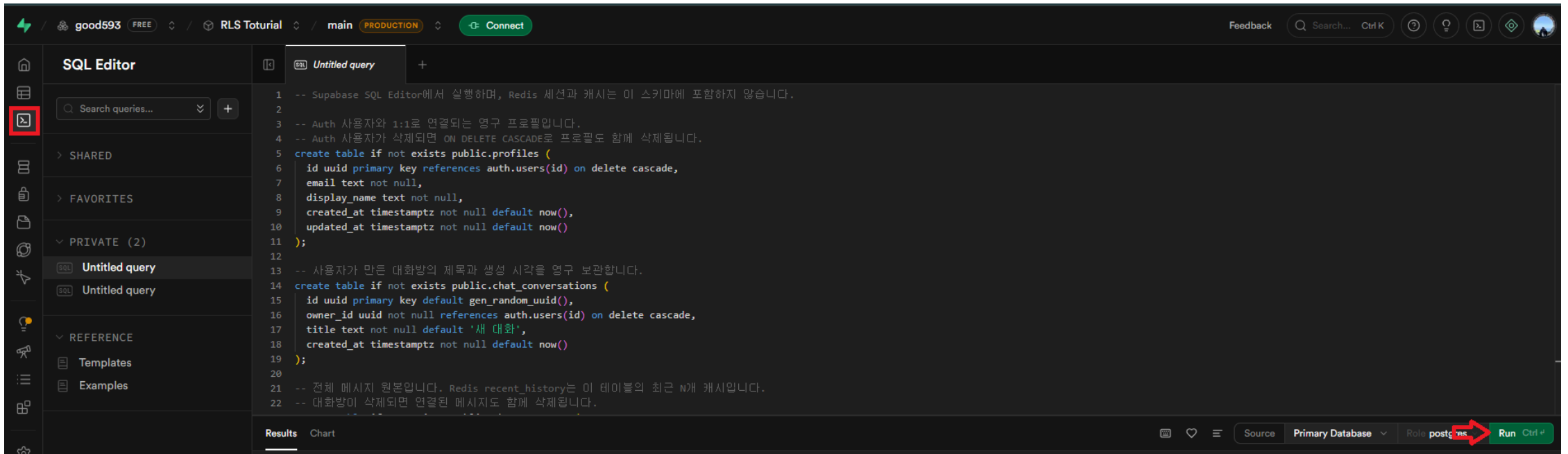
```
list(reversed(...)) → [msg1, msg2, msg3, msg4, msg5]
```

R PUSH 순서대로 넣으면:

```
List: [msg1, msg2, msg3, msg4, msg5] ← 올바른 순서
```

8. 서버 실행 및 Swagger 실습

Supabase에 schema.sql 실행 (기존 테이블 삭제한 후 실행)



가상환경 생성

```
# pyproject.toml과 uv.lock 기준으로 의존성 설치  
uv sync
```

서버 실행

```
# 1. .env 파일에 실제 키 입력  
# 2. 서버 실행  
uv run uvicorn main:app --reload --port 8002
```

8001 포트는 Docker Compose의 RedisInsight가 사용하므로 FastAPI는 8002 포트
실행합니다.

Swagger UI 실습 (<http://localhost:8002/docs>)

각 API에서 Try it out을 누른 뒤 아래 순서대로 입력합니다.

이번 실습은 Swagger UI 우측 상단의 **Authorize** 기능을 사용하지 않습니다.

로그인 응답으로 받은 `session_id` 를 보호된 API의 `X-Session-Id` 헤더에 직접 입력합니다.

실습 1. Redis 연결 상태 확인

GET /health → Try it out → Execute

GET

/health Health

^

Parameters

Cancel

No parameters

Execute

→ 200 OK 확인

Redis 연결 오류가 발생하면 `docker compose ps` 와
`docker compose exec redis redis-cli ping` 을 먼저 확인합니다.

Code	Details
200	Response body <pre>{ "redis": true, "supabase_url": "https://jnhsvdkrunsyxjlvkiix.supabase.co" }</pre>  Download Response headers <pre>content-length: 72 content-type: application/json date: Sun, 21 Jun 2026 12:57:44 GMT server: uvicorn</pre>

실습 2. 사용자 A 회원가입

POST /auth/signup → Try it out → Body 입력 → Execute

```
{  
  "email": "swagger08-a@example.com",  
  "password": "swagger1234",  
  "display_name": "사용자 A"  
}
```

→ 200 OK 확인

→ 응답의 user_id, email, email_confirm_required 확인

email_confirm_required 가 true 이면 이메일 인증을 완료합니다. 또는 Supabase Auth 설정에서 Confirm email을 비활성화한 뒤, 기존 사용자를 삭제하고 새로운 이메일로 다시 가입합니다.

POST

/auth/signup Signup

^

Parameters

Cancel

Reset

No parameters

Request body required

application/json

▼

Edit Value | Schema

```
{  "email": "swagger08-a@example.com",  "password": "swagger1234",  "display_name": "사용자 A"}

```

//

Execute

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8002/auth/signup' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "email": "swagger08-a@example.com",
    "password": "swagger1234",
    "display_name": "사용자 A"
  }'
```



Request URL

`http://localhost:8002/auth/signup`

Server response

Code

Details

200

Response body

```
{
  "user_id": "39d6cb59-57d1-424f-a957-b8290e653d8c",
  "email": "swagger08-a@example.com",
  "email_confirm_required": false
}
```



Download

Response headers

```
content-length: 115
content-type: application/json
date: Sun, 21 Jun 2026 12:59:18 GMT
server: uvicorn
```

Supabase에서 저장된 사용자 확인

The screenshot shows the Supabase Table Editor interface. The browser address bar displays the URL: `supabase.com/dashboard/project/jnhsvdkrunsyxjlvkiix/editor/17670?schema=public`. The interface includes a sidebar with navigation icons and a main table view.

Table Editor

Schema: `public`

Filter by id, email, display_name... or ask AI

	<code>id</code> uuid	<code>email</code> text	<code>display_name</code> text	<code>created_at</code> timestampz	<code>updated_at</code> timestampz
	<code>39d6cb59-57d1-424f-a957-b8290...</code>	<code>swagger08-a@example.com</code>	사용자 A	<code>2026-06-21 12:59:21.64739+00</code>	<code>2026-06-21 12:59:21.64739+00</code>

실습 3. 사용자 A 로그인 및 앱 세션 저장

POST /auth/login → Try it out → Body 입력 → Execute

```
{  
  "email": "swagger08-a@example.com",  
  "password": "swagger1234"  
}
```

POST

/auth/login

Login

^

Parameters

Cancel

Reset

No parameters

Request body

required

application/json

▼

Edit Value

Schema

```
{
  "email": "swagger08-a@example.com",
  "password": "swagger1234"
}
```

Execute

- 200 OK 확인
- 응답의 `user_id` 를 `USER_A_USER_ID` 로 저장
- 응답의 `session_id` 를 `USER_A_SESSION_ID` 로 저장
- `ttl_seconds` 가 `SESSION_TTL_SECONDS` 와 비슷한 값인지 확인

```
{  
  "session_id": "발급된-session-id",  
  "user_id": "사용자-A-UUID",  
  "email": "swagger08-a@example.com",  
  "ttl_seconds": 3600  
}
```

Supabase의 `access_token` 과 `refresh_token` 은 Redis에 보관되며 클라이언트에는 앱의 `session_id` 만 반환됩니다.

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8002/auth/login' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "email": "swagger08-a@example.com",
    "password": "swagger1234"
  }'
```



Request URL

```
http://localhost:8002/auth/login
```

Server response

Code

Details

200

Response body

```
{
  "session_id": "4dcdb429-7136-460f-a8aa-b353f71a8486",
  "user_id": "39d6cb59-57d1-424f-a957-b8290e653d8c",
  "email": "swagger08-a@example.com",
  "ttl_seconds": 3600
}
```

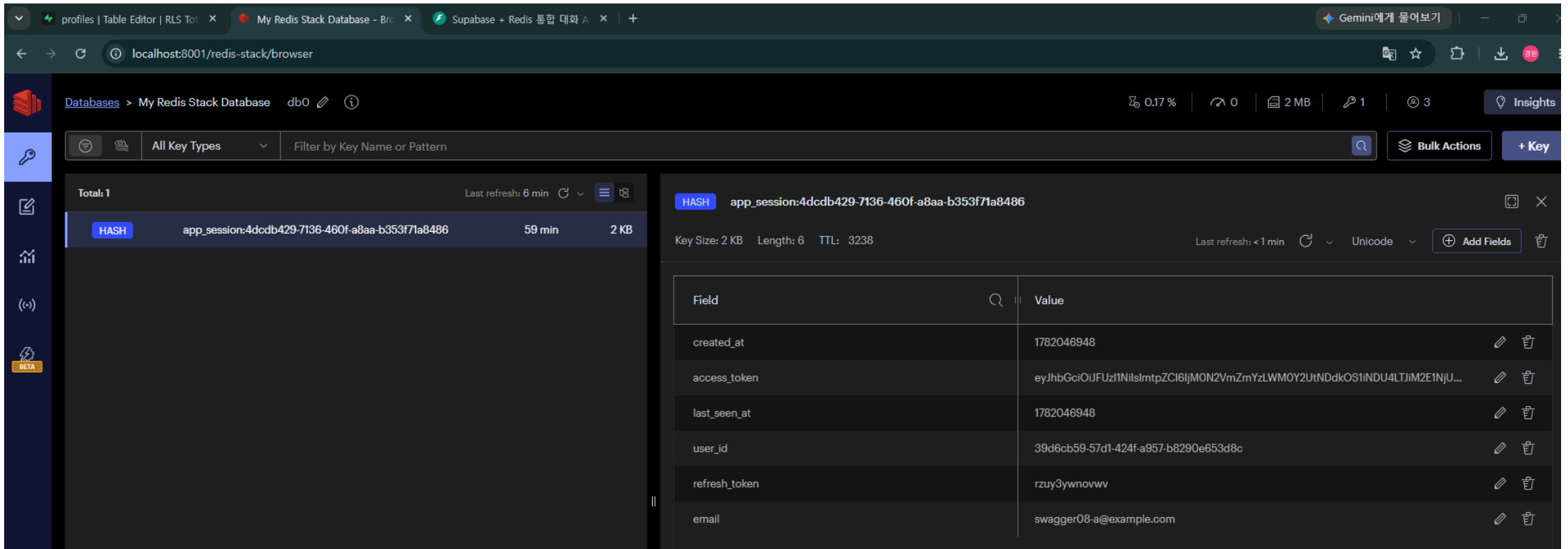


Download

Response headers

```
content-length: 155
content-type: application/json
date: Sun, 21 Jun 2026 13:02:26 GMT
server: uvicorn
```

Redis에서 저장된 token 확인



The screenshot displays the Supabase Redis Stack Database browser interface. The top navigation bar shows the database name 'My Redis Stack Database' and the instance 'db0'. The main panel is divided into two sections: a left sidebar showing a list of keys and a right panel showing the details of a selected key.

Left Sidebar (Key List):

- Total: 1
- Last refresh: 6 min
- Key Type: HASH
- Key Name: app_session:4dcdb429-7136-460f-a8aa-b353f71a8486
- Size: 59 min
- Size: 2 KB

Right Panel (Key Details):

Key Name: app_session:4dcdb429-7136-460f-a8aa-b353f71a8486

Key Size: 2 KB | Length: 6 | TTL: 3238

Last refresh: < 1 min

Unicode

+ Add Fields

Field	Value
created_at	1782046948
access_token	eyJhbGciOiJIUzU1NiIsImtpZCI6IjMON2VmZmZlWMOY2UtNDdkOS1lNDU4LTJiM2E1NjU...
last_seen_at	1782046948
user_id	39d6cb59-57d1-424f-a957-b8290e653d8c
refresh_token	rzuy3ywnovvw
email	swagger08-a@example.com

실습 4. 세션 없이 보호된 API 요청

GET /me → Try it out → X-Session-Id 를 비운 상태로 Execute

GET

/me Me

^

Parameters

Cancel

Name	Description
X-Session-Id	X-Session-Id
string (string null)	
(header)	

Execute

→ 401 Unauthorized 확인

Code	Details
401 <i>Undocumented</i>	Error: Unauthorized Response body <pre>{ "detail": "X-Session-Id 헤더가 필요합니다" }</pre> Response headers <pre>content-length: 51 content-type: application/json date: Sun, 21 Jun 2026 13:10:34 GMT server: uvicorn</pre>

실습 5. 사용자 A 프로필과 세션 TTL 조회

GET /me → Try it out

1. X-Session-Id 에 USER_A_SESSION_ID 입력
2. Execute 클릭

The screenshot shows a REST client interface with a light blue header bar. On the left, a blue button labeled 'GET' is followed by the path '/me' and the text 'Me'. On the right side of the header bar is an upward-pointing chevron icon. Below the header bar is a section titled 'Parameters' with a red 'Cancel' button in the top right corner. Under 'Parameters', there is a table with two columns: 'Name' and 'Description'. The table contains one entry: 'X-Session-Id' with a description of '4dcdb429-7136-460f-a8aa-b353f71a8486'. To the left of the description, there is a text box containing the same ID. Below the table, there is a code snippet:

```
string |  
(string |  
null)  
(header)
```

. At the bottom of the interface is a large blue button labeled 'Execute'.

Name	Description
X-Session-Id	4dcdb429-7136-460f-a8aa-b353f71a8486

```
string |  
(string |  
null)  
(header)
```

Execute

- 200 OK 확인
- profile.email 이 USER_A_EMAIL 과 같은지 확인
- session_ttl_seconds 가 갱신되었는지 확인

보호된 API를 정상 호출할 때마다 Redis 세션의 TTL이 다시 설정되는 Sliding Expiry 방식입니다.

Code

Details

200

Response body

```
{
  "profile": {
    "id": "39d6cb59-57d1-424f-a957-b8290e653d8c",
    "email": "swagger08-a@example.com",
    "display_name": "사용자 A",
    "created_at": "2026-06-21T12:59:21.64739+00:00",
    "updated_at": "2026-06-21T12:59:21.64739+00:00"
  },
  "session_ttl_seconds": 3599
}
```

 Download

Response headers

```
content-length: 241
content-type: application/json
date: Sun, 21 Jun 2026 13:12:19 GMT
server: uvicorn
```

실습 6. 사용자 A 대화방 생성

POST /conversations → Try it out

1. X-Session-Id 에 USER_A_SESSION_ID 입력
2. Body 입력
3. **Execute** 클릭

```
{  
  "title": "Redis 세션 실습 대화"  
}
```

POST

/conversations

Create Conversation

⌵

Parameters

Cancel

Reset

Name	Description
X-Session-Id	4dcdb429-7136-460f-a8aa-b353f71a8486
<code>string (string null) (header)</code>	

Request body

required

application/json

⌵

Edit Value

Schema

```
{  "title": "Redis 세션 실습 대화"}  
```

Execute

- 201 Created 확인
- 응답의 id 를 USER_A_CONVERSATION_ID 로 저장
- 응답의 owner_id 가 USER_A_USER_ID 와 같은지 확인

Code	Details
201	Response body <pre>{ "id": "4514c0f9-0919-4f89-94fb-41d78fc33643", "owner_id": "39d6cb59-57d1-424f-a957-b8290e653d8c", "title": "Redis 세션 실습 대화", "created_at": "2026-06-21T13:13:56.555524+00:00" }</pre>  Download Response headers <pre>content-length: 180 content-type: application/json date: Sun, 21 Jun 2026 13:13:55 GMT server: uvicorn</pre>

실습 7. 사용자 A 대화방 목록 조회

GET /conversations → Try it out

1. X-Session-Id 에 USER_A_SESSION_ID 입력
2. Execute 클릭

GET /conversations List Conversations

Parameters

Cancel

Name	Description
X-Session-Id	4dcdb429-7136-460f-a8aa-b353f71a8486
string (string null) {header}	

Execute

- 200 OK 확인
- USER_A_CONVERSATION_ID 가 목록에 있는지 확인
- 반환된 모든 행의 owner_id 가 USER_A_USER_ID 와 같은지 확인

Code	Details
200	Response body <pre>[{ "id": "4514c0f9-0919-4f89-94fb-41d78fc33643", "owner_id": "39d6cb59-57d1-424f-a957-b8290e653d8c", "title": "Redis 세션 실습 대화", "created_at": "2026-06-21T13:13:56.555524+00:00" }]</pre>  Download Response headers <pre>content-length: 182 content-type: application/json date: Sun, 21 Jun 2026 13:15:26 GMT server: uvicorn</pre>

실습 8. Supabase와 Redis에 메시지 저장

POST /conversations/{conversation_id}/messages → Try it out

1. conversation_id 에 USER_A_CONVERSATION_ID 입력
2. X-Session-Id 에 USER_A_SESSION_ID 입력
3. 다음 Body를 입력하고 Execute 클릭

```
{  
  "role": "user",  
  "content": "Supabase와 Redis의 역할 차이를 설명해 줘."  
}
```

Parameters

CancelReset

Name	Description
conversation_id ★ required string <i>(path)</i>	4514c0f9-0919-4f89-94fb-41d78fc33643
X-Session-Id string (string null) <i>(header)</i>	4dcdb429-7136-460f-a8aa-b353f71a8486

Request body required

application/json

Edit Value | Schema

```
{  "role": "user",  "content": "Supabase와 Redis의 역할 차이를 설명해 줘."}
```

→ 201 Created 확인

→ 응답의 conversation_id 와 owner_id 확인

Code	Details
201	Response body <pre>{ "id": "47299095-75e4-45c0-93fd-d2041cb9f4c6", "conversation_id": "4514c0f9-0919-4f89-94fb-41d78fc33643", "owner_id": "39d6cb59-57d1-424f-a957-b8290e653d8c", "role": "user", "content": "Supabase와 Redis의 역할 차이를 설명해 줘.", "created_at": "2026-06-21T13:17:24.82142+00:00" }</pre>  Download Response headers <pre>content-length: 278 content-type: application/json date: Sun, 21 Jun 2026 13:17:23 GMT server: uvicorn</pre>

같은 API에서 두 번째 메시지도 저장합니다.

```
{  
  "role": "assistant",  
  "content": "Supabase는 원본을 영구 저장하고 Redis는 최근 데이터를 캐시합니다."  
}
```

메시지는 Supabase에 먼저 저장된 뒤 Redis의 최근 대화 List에 추가됩니다.

role에는 user, assistant, system 중 하나만 입력할 수 있습니다.

POST

/conversations/{conversation_id}/messages

Create Message

^

Parameters

Cancel

Reset

Name	Description
conversation_id ★ required string <i>(path)</i>	4514c0f9-0919-4f89-94fb-41d78fc33643
X-Session-Id string (string null) <i>(header)</i>	4dcdb429-7136-460f-a8aa-b353f71a8486

Request body required

application/json

Edit Value | Schema

```
{  "role": "assistant",  "content": "Supabase는 원본을 영구 저장하고 Redis는 최근 데이터를 캐시합니다."}
```

Execute

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8002/conversations/4514c0f9-0919-4f89-94fb-41d78fc33643/messages' \
  -H 'accept: application/json' \
  -H 'X-Session-Id: 4dcdb429-7136-460f-a8aa-b353f71a8486' \
  -H 'Content-Type: application/json' \
  -d '{
    "role": "assistant",
    "content": "Supabase는 원본을 영구 저장하고 Redis는 최근 데이터를 캐시합니다."
  }'
```

Request URL

```
http://localhost:8002/conversations/4514c0f9-0919-4f89-94fb-41d78fc33643/messages
```

Server response

Code

Details

201

Response body

```
{
  "id": "a14934bd-a860-43c4-a88b-0b3882abaca0",
  "conversation_id": "4514c0f9-0919-4f89-94fb-41d78fc33643",
  "owner_id": "39d6cb59-57d1-424f-a957-b8290e653d8c",
  "role": "assistant",
  "content": "Supabase는 원본을 영구 저장하고 Redis는 최근 데이터를 캐시합니다.",
  "created_at": "2026-06-21T13:19:05.840721+00:00"
}
```



Download

Response headers

```
content-length: 319
content-type: application/json
date: Sun, 21 Jun 2026 13:19:05 GMT
server: uvicorn
```

Redis Stack Database 확인

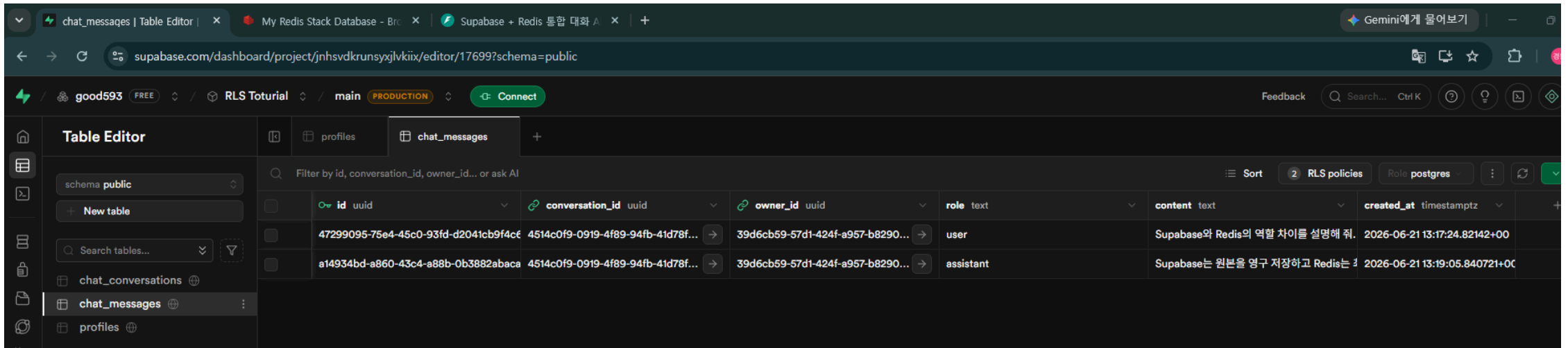
The screenshot displays the Redis Stack Database browser interface. The top navigation bar shows the current database is 'db0'. The main panel lists two keys:

- app_session:4dcd429-7136-460f-a8aa-b353f71a8486** (HASH type, 59 min TTL, 2 KB size)
- recent_history:39d6cb59-57d1-424f-a957-b8290e653d8c:4514c0f9-0919-4f89-94fb-41d78fc33643** (LIST type, 23 h TTL, 776 B size)

The right-hand panel provides a detailed view of the selected LIST key. It shows the key size (776 B), length (2), and TTL (86370). Below this, a table displays the elements of the list:

Index	Element
0	{"id": "47299095-75e4-45c0-93fd-d2041cb9f4c6", "conversation_id": "4514c0f9-0919-4f89-94fb-41d78fc33643"...
1	{"id": "a14934bd-a860-43c4-a88b-0b3882abaca0", "conversation_id": "4514c0f9-0919-4f89-94fb-41d78fc3364"...

Supabase Database 확인



The screenshot shows the Supabase Table Editor interface for the 'chat_messages' table in the 'public' schema. The table has the following columns: id (uuid), conversation_id (uuid), owner_id (uuid), role (text), content (text), and created_at (timestampz). There are two rows of data displayed.

id	conversation_id	owner_id	role	content	created_at
47299095-75e4-45c0-93fd-d2041cb9f4c6	4514c0f9-0919-4f89-94fb-41d78f...	39d6cb59-57d1-424f-a957-b8290...	user	Supabase와 Redis의 역할 차이를 설명해 줘.	2026-06-21 13:17:24.82142+00
a14934bd-a860-43c4-a88b-0b3882abaca	4514c0f9-0919-4f89-94fb-41d78f...	39d6cb59-57d1-424f-a957-b8290...	assistant	Supabase는 원본을 영구 저장하고 Redis는 캐시	2026-06-21 13:19:05.840721+00

실습 9. 전체 이력과 최근 캐시 비교

먼저 `GET /conversations/{conversation_id}/messages` 를 실행합니다.

1. `conversation_id` 에 `USER_A_CONVERSATION_ID` 입력
2. `X-Session-Id` 에 `USER_A_SESSION_ID` 입력
3. **Execute** 클릭

The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** /conversations/{conversation_id}/messages
- Parameters:**

Name	Description
conversation_id ★ required string (path)	4514c0f9-0919-4f89-94fb-41d78fc33643
X-Session-Id string (string null) (header)	4dcdb429-7136-460f-a8aa-b353f71a8486
- Buttons:** Cancel (red), Execute (blue)

Code

Details

200

Response body

```
[
  {
    "id": "47299095-75e4-45c0-93fd-d2041cb9f4c6",
    "conversation_id": "4514c0f9-0919-4f89-94fb-41d78fc33643",
    "owner_id": "39d6cb59-57d1-424f-a957-b8290e653d8c",
    "role": "user",
    "content": "Supabase와 Redis의 역할 차이를 설명해 줘.",
    "created_at": "2026-06-21T13:17:24.82142+00:00"
  },
  {
    "id": "a14934bd-a860-43c4-a88b-0b3882abaca0",
    "conversation_id": "4514c0f9-0919-4f89-94fb-41d78fc33643",
    "owner_id": "39d6cb59-57d1-424f-a957-b8290e653d8c",
    "role": "assistant",
    "content": "Supabase는 원본을 영구 저장하고 Redis는 최근 데이터를 캐시합니다.",
    "created_at": "2026-06-21T13:19:05.840721+00:00"
  }
]
```



Download

Response headers

```
content-length: 600
content-type: application/json
date: Sun, 21 Jun 2026 13:22:56 GMT
server: uvicorn
```

→ Supabase에 저장된 전체 메시지가 시간순으로 반환되는지 확인

이어서 `GET /conversations/{conversation_id}/recent` 를 같은 값으로 실행합니다.

GET

/conversations/{conversation_id}/recent Recent Messages

^

Parameters

Cancel

Name	Description
conversation_id ★ required string <i>(path)</i>	4514c0f9-0919-4f89-94fb-41d78fc33643
X-Session-Id string (string null) <i>(header)</i>	4dcdb429-7136-460f-a8aa-b353f71a8486

Execute

Code

Details

200

Response body

```
[
  {
    "id": "47299095-75e4-45c0-93fd-d2041cb9f4c6",
    "conversation_id": "4514c0f9-0919-4f89-94fb-41d78fc33643",
    "owner_id": "39d6cb59-57d1-424f-a957-b8290e653d8c",
    "role": "user",
    "content": "Supabase와 Redis의 역할 차이를 설명해 줘.",
    "created_at": "2026-06-21T13:17:24.82142+00:00"
  },
  {
    "id": "a14934bd-a860-43c4-a88b-0b3882abaca0",
    "conversation_id": "4514c0f9-0919-4f89-94fb-41d78fc33643",
    "owner_id": "39d6cb59-57d1-424f-a957-b8290e653d8c",
    "role": "assistant",
    "content": "Supabase는 원본을 영구 저장하고 Redis는 최근 데이터를 캐시합니다.",
    "created_at": "2026-06-21T13:19:05.840721+00:00"
  }
]
```



Download

Response headers

```
content-length: 600
content-type: application/json
date: Sun, 21 Jun 2026 13:24:56 GMT
server: uvicorn
```

실습 10. Redis 최근 캐시 삭제

RedisInsight의 Browser에서 다음 키를 찾습니다.

```
recent_history:{USER_A_USER_ID}:{USER_A_CONVERSATION_ID}
```

1. 해당 키를 삭제

The screenshot shows the RedisInsight Browser interface. On the left, a list of keys is displayed. The key `recent_history:39d6cb59-57d1-424f-a957-b8290e653...` is highlighted, and a red box is drawn around its delete icon (a trash can). A red arrow points from this icon to the right-hand pane. The right-hand pane shows the details of the selected key, including its type (LIST), size (776 B), length (2), and TTL (85939). Below this, a table lists the elements of the list.

Index	Element
0	<code>{"id": "47299095-75e4-45c0-93fd-d2041cb9f4c6", "conversation_id": "4514c0f9-0919-4f89-94fb-41d78fc33643", "owner..."}</code>
1	<code>{"id": "a14934bd-a860-43c4-a88b-0b3882abaca0", "conversation_id": "4514c0f9-0919-4f89-94fb-41d78fc33643", "own..."}</code>

2. Swagger UI로 돌아와 `GET /conversations/{conversation_id}/recent` 실행

3. `conversation_id` 와 `X-Session-Id` 에 사용자 A의 값 입력

GET

/conversations/{conversation_id}/recent

Recent Messages

^

Parameters

Cancel

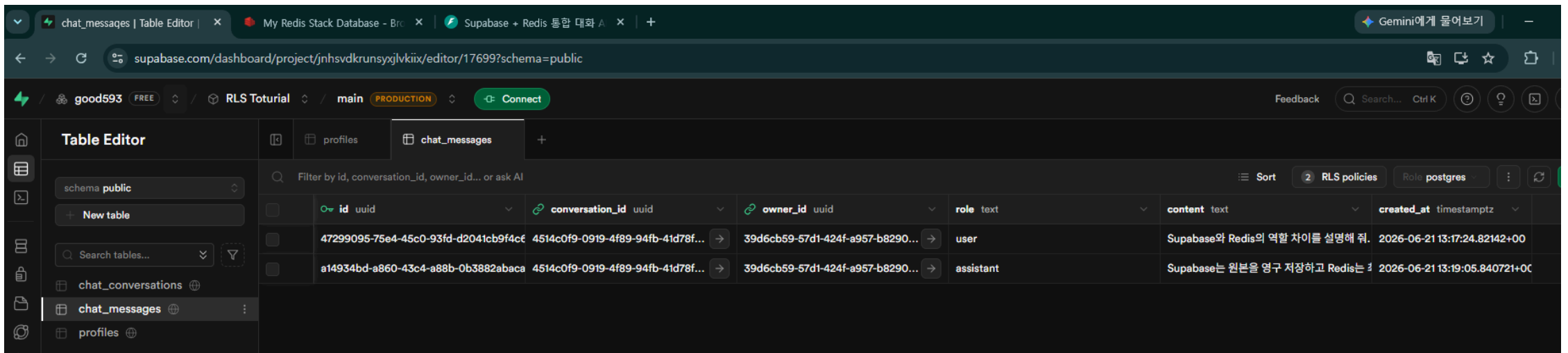
Name	Description
conversation_id ★ required string <i>(path)</i>	4514c0f9-0919-4f89-94fb-41d78fc33643
X-Session-Id string (string null) <i>(header)</i>	4dcdb429-7136-460f-a8aa-b353f71a8486

Execute

→ 200 OK 와 빈 배열 [] 확인

Code	Details
200	Response body []  Download Response headers content-length: 2 content-type: application/json date: Sun, 21 Jun 2026 13:30:38 GMT server: uvicorn

Redis 캐시만 삭제했으므로 Supabase의 원본 메시지는 그대로 남아 있습니다.



The screenshot shows the Supabase Table Editor for the 'chat_messages' table in the 'public' schema. The table has columns: id, conversation_id, owner_id, role, content, and created_at. There are two rows of data.

id	conversation_id	owner_id	role	content	created_at
47299095-75e4-45c0-93fd-d2041cb9f4c6	4514c0f9-0919-4f89-94fb-41d78f...	39d6cb59-57d1-424f-a957-b8290...	user	Supabase와 Redis의 역할 차이를 설명해 줘.	2026-06-21 13:17:24.82142+00
a14934bd-a860-43c4-a88b-0b3882abaca	4514c0f9-0919-4f89-94fb-41d78f...	39d6cb59-57d1-424f-a957-b8290...	assistant	Supabase는 원본을 영구 저장하고 Redis는 캐시	2026-06-21 13:19:05.840721+00

실습 11. Supabase 원본으로 캐시 복구

POST /conversations/{conversation_id}/recent/rebuild → Try it out

1. conversation_id 에 USER_A_CONVERSATION_ID 입력
2. X-Session-Id 에 USER_A_SESSION_ID 입력
3. Execute 클릭

POST /conversations/{conversation_id}/recent/rebuild Rebuild Recent Messages

Parameters

Cancel

Name	Description
conversation_id ★ required string <i>(path)</i>	4514c0f9-0919-4f89-94fb-41d78fc33643
X-Session-Id string (string null) <i>(header)</i>	4dcdb429-7136-460f-a8aa-b353f71a8486

Execute

- 200 OK 확인
- rebuilt 가 2 인지 확인
- recent_key 가 실습 10에서 삭제한 키와 같은지 확인

Code	Details
200	Response body <pre>{ "rebuilt": 2, "recent_key": "recent_history:39d6cb59-57d1-424f-a957-b8290e653d8c:4514c0f9-0919-4f89-94fb-41d78fc33643" }</pre>  Download Response headers <pre>content-length: 117 content-type: application/json date: Sun, 21 Jun 2026 13:33:26 GMT server: uvicorn</pre>

다시 GET /conversations/{conversation_id}/recent 를 실행합니다.
→ 두 메시지가 오래된 것부터 올바른 순서로 복구되었는지 확인

GET

/conversations/{conversation_id}/recent Recent Messages

^

Parameters

Cancel

Name	Description
conversation_id ★ required string <i>(path)</i>	4514c0f9-0919-4f89-94fb-41d78fc33643
X-Session-Id string (string null) <i>(header)</i>	4dcdb429-7136-460f-a8aa-b353f71a8486

Execute

Code

Details

200

Response body

```
[
  {
    "id": "47299095-75e4-45c0-93fd-d2041cb9f4c6",
    "conversation_id": "4514c0f9-0919-4f89-94fb-41d78fc33643",
    "owner_id": "39d6cb59-57d1-424f-a957-b8290e653d8c",
    "role": "user",
    "content": "Supabase와 Redis의 역할 차이를 설명해 줘.",
    "created_at": "2026-06-21T13:17:24.82142+00:00"
  },
  {
    "id": "a14934bd-a860-43c4-a88b-0b3882abaca0",
    "conversation_id": "4514c0f9-0919-4f89-94fb-41d78fc33643",
    "owner_id": "39d6cb59-57d1-424f-a957-b8290e653d8c",
    "role": "assistant",
    "content": "Supabase는 원본을 영구 저장하고 Redis는 최근 데이터를 캐시합니다.",
    "created_at": "2026-06-21T13:19:05.840721+00:00"
  }
]
```



Download

Response headers

```
content-length: 600
content-type: application/json
date: Sun, 21 Jun 2026 13:34:53 GMT
server: uvicorn
```

Redis Stack Database에 데이터가 생성된 것 확인

The screenshot shows the Redis Stack Database browser interface. The left sidebar contains navigation icons for databases, keys, and insights. The main area displays a list of keys under the 'My Redis Stack Database' instance. The 'recent_history' key is selected, and its details are shown on the right.

Key List:

Key Type	Key Name	TTL	Size
HASH	app_session:4dcdb429-7136-460f-a8aa-b353f71a8486	59 min	2 KB
LIST	recent_history:39d6cb59-57d1-424f-a957-b8290e653d8c:45	23 h	776 B

Key Details:

Key Name: recent_history:39d6cb59-57d1-424f-a957-b8290e653d8c:4514c0f9-0919-4f89-94fb-41d78fc33643

Key Size: 776 B | Length: 2 | TTL: 86366

Last refresh: 1 min

Unicode

Elements:

Index	Element
0	{"id": "47299095-75e4-45c0-93fd-d2041cb9f4c6", "conversation_id": "4514c0f9-0919-4f89-94fb-41d78fc33643", "..."}
1	{"id": "a14934bd-a860-43c4-a88b-0b3882abaca0", "conversation_id": "4514c0f9-0919-4f89-94fb-41d78fc33643", "..."}

실습 12. 로그아웃 후 세션 무효화

POST /auth/logout → Try it out

1. X-Session-Id 에 USER_A_SESSION_ID 입력
2. Execute 클릭

POST /auth/logout Logout

Parameters

Cancel

Name	Description
X-Session-Id	4dcdb429-7136-460f-a8aa-b353f71a8486
string (string null) {header}	

Execute

→ 200 OK 와 "deleted": true 확인

Code	Details
200	Response body <pre>{ "session_id": "4dcdb429-7136-460f-a8aa-b353f71a8486", "deleted": true }</pre>  Download Response headers <pre>content-length: 68 content-type: application/json date: Sun, 21 Jun 2026 13:38:21 GMT server: unicorn</pre>

같은 `USER_A_SESSION_ID` 로 `GET /me` 를 다시 실행합니다.

GET

/me Me

^

Parameters

Cancel

Name	Description
X-Session-Id	4dcdb429-7136-460f-a8aa-b353f71a8486
string (string null) (header)	

Execute

→ 401 Unauthorized 확인

Code	Details
401 <i>Undocumented</i>	Error: Unauthorized Response body <pre>{ "detail": "앱 세션이 없거나 만료되었습니다" }</pre> Response headers <pre>content-length: 58 content-type: application/json date: Sun, 21 Jun 2026 13:39:53 GMT server: uvicorn</pre>